TCP

client.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char message[BUFFER_SIZE];
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }
    if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
    printf("Connected to server. Type 'exit' to quit.\n");
    while(1) {
        printf("You: ");
        fgets(message, BUFFER_SIZE, stdin);
        message[strcspn(message, "\n")] = 0;
        send(sock, message, strlen(message), 0);
        if (strncmp(message, "exit", 4) == 0) {
            printf("Exiting...\n");
            break;
        }
        memset(buffer, 0, BUFFER_SIZE);
        int valread = read(sock, buffer, BUFFER_SIZE);
        printf("Server: %s\n", buffer);
    }
    close(sock);
    return 0;
}
```

server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    char *response = "Message received by server";
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
```

```
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    printf("Server listening on port %d...\n", PORT);
    if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }
    while(1) {
        memset(buffer, 0, BUFFER_SIZE);
        int valread = read(new_socket, buffer, BUFFER_SIZE);
        if (valread <= 0) {
            printf("Client disconnected.\n");
            break;
        }
        printf("Client: %s\n", buffer);
        send(new_socket, response, strlen(response), 0);
        printf("Response sent to client.\n");
        if (strncmp(buffer, "exit", 4) == 0) {
            printf("Exit signal received. Shutting down.\n");
            break;
        }
    }
    close(new_socket);
    close(server_fd);
    return 0;
}
client O/P
Connected to server. Type 'exit' to quit.
You: Hi
Server: Message received by server
You: Hello
Server: Message received by server
You: How are you?
Server: Message received by server
You: exit
Exiting...
```

```
server O/P
Server listening on port 8080...
Client: Hi
Response sent to client.
Client: Hello
Response sent to client.
Client: How are you?
Response sent to client.
Client: exit
Response sent to client.
Exit signal received. Shutting down.
```

UDP

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_PORT 12345
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr;
    socklen_t addr_len = sizeof(server_addr);
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(SERVER_PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    while (1) {
        printf("Enter message: ");
        fgets(buffer, BUFFER_SIZE, stdin);
        buffer[strcspn(buffer, "\n")] = 0;
```

```
        sendto(sockfd, buffer, strlen(buffer), 0, (struct
sockaddr *)&server_addr, addr_len);
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting client...\n");
            break;
        }
        int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, NULL,
NULL);
        if (n < 0) {
            perror("Receive failed");
            continue;
        }
        buffer[n] = '\0';
        printf("Server response: %s\n", buffer);
    }
    close(sockfd);
    return 0;
}
```

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
#define SERVER_PORT 12345
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr, client_addr;
    socklen_t client_addr_len = sizeof(client_addr);
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(SERVER_PORT);
    if (bind(sockfd, (const struct sockaddr *)&server_addr,
sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }
    printf("UDP Server listening on port %d...\n", SERVER_PORT);
    while (1) {
        int n = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct
sockaddr *)&client_addr, &client_addr_len);
        if (n < 0) {
            perror("Receive failed");
            continue;
        }
        buffer[n] = '\0';
        if (strcmp(buffer, "exit") == 0) {
            printf("Exit Signal Received, Shutting down
server...\n");
            break;
        }
        printf("Received from client: %s\n", buffer);
        char response[BUFFER_SIZE];
        printf("Enter message: ");
        fgets(response, BUFFER_SIZE, stdin);
        response[strcspn(response, "\n")] = 0;
        sendto(sockfd, response, strlen(response), 0, (struct
sockaddr *)&client_addr, client_addr_len);
    }
    close(sockfd);
    return 0;
}
client O/P
Enter message: Hi
Server response: Hello
Enter message: How are you?
Server response: Fine
Enter message: exit
Exiting client...

server O/P
UDP Server listening on port 12345...
Received from client: Hi
Enter message: Hello
```

```
Received from client: How are you?
Enter message: Fine
Exit Signal Received, Shutting down server...
```

Sliding Window Protocols

Go Back N

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

int main() {
    int window_size, total_frames, frames_sent = 0;
    int current_frame = 1;
    srand(time(NULL));
    printf("--- Go-Back-N Protocol Simulation ---\n");
    printf("Enter window size: ");
    scanf("%d", &window_size);
    printf("Enter total number of frames to transmit: ");
    scanf("%d", &total_frames);
    printf("-----------------------------------\n\n");
    while (current_frame <= total_frames) {
        int successful_acks = 0;
        for (int k = current_frame; k < current_frame +
window_size && k <= total_frames; k++) {
            printf("Sending Frame %d...\n", k);
            frames_sent++;
        }
        for (int k = current_frame; k < current_frame +
window_size && k <= total_frames; k++) {
            int ack_status = rand() % 5;
            sleep(1);
            if (ack_status != 0) {
                printf("Acknowledgment for Frame %d received.
\n", k);
                successful_acks++;
            } else {
                printf("--> ERROR: Acknowledgment for Frame %d
lost or corrupted!\n", k);
                printf("--> Action: Go-Back-N triggered.
Retransmitting from Frame %d...\n", k);
                break;
            }
        }
        printf("\n");
        current_frame = current_frame + successful_acks;
    }
    printf("-----------------------------------\n");
    printf("Simulation Complete.\n");
    printf("Total original frames: %d\n", total_frames);
    printf("Total frames transmitted (including
retransmissions): %d\n", frames_sent);
    return 0;
}
```

O/P

```
--- Go-Back-N Protocol Simulation ---
Enter window size: 4
Enter total number of frames to transmit: 10
-----------------------------------

Sending Frame 1...
Sending Frame 2...
Sending Frame 3...
Sending Frame 4...
Acknowledgment for Frame 1 received.
Acknowledgment for Frame 2 received.
Acknowledgment for Frame 3 received.
--> ERROR: Acknowledgment for Frame 4 lost or corrupted!
--> Action: Go-Back-N triggered. Retransmitting from Frame 4...

Sending Frame 4...
Sending Frame 5...
Sending Frame 6...
Sending Frame 7...
Acknowledgment for Frame 4 received.
--> ERROR: Acknowledgment for Frame 5 lost or corrupted!
--> Action: Go-Back-N triggered. Retransmitting from Frame 5...

Sending Frame 5...
Sending Frame 6...
```

```
Sending Frame 7...
Sending Frame 8...
Acknowledgment for Frame 5 received.
Acknowledgment for Frame 6 received.
Acknowledgment for Frame 7 received.
Acknowledgment for Frame 8 received.

Sending Frame 9...
Sending Frame 10...
Acknowledgment for Frame 9 received.
Acknowledgment for Frame 10 received.

-------------------------------------
Simulation Complete.
Total original frames: 10
Total frames transmitted (including retransmissions): 14
```

Selective Repeat

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#define NOT_SENT 0
#define SENT 1
#define ACKED 2

int main() {
    int window_size, total_frames, frames_sent_count = 0;
    int base = 1;
    srand(time(NULL));
    printf("--- Selective Repeat Protocol Simulation ---\n");
    printf("Enter window size: ");
    scanf("%d", &window_size);
    printf("Enter total number of frames to transmit: ");
    scanf("%d", &total_frames);
    printf("-----------------------------------------\n\n");
    int *frames = (int *)calloc(total_frames + 1, sizeof(int));
    if (frames == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
    while (base <= total_frames) {
        for (int i = base; i < base + window_size && i <=
total_frames; i++) {
            if (frames[i] == NOT_SENT) {
                printf("Sending Frame %d...\n", i);
                frames[i] = SENT;
                frames_sent_count++;
            }
        }
        for (int i = base; i < base + window_size && i <=
total_frames; i++) {
            if (frames[i] == SENT) {
                int ack_status = rand() % 5;
                sleep(1);

                if (ack_status != 0) {
                    printf("Acknowledgment for Frame %d
received.\n", i);
                    frames[i] = ACKED;
                } else {
                    printf("--> ERROR: Acknowledgment for Frame
%d lost or corrupted!\n", i);
                    printf("--> Action: Tagging Frame %d for
Selective Retransmission.\n", i);
                    frames[i] = NOT_SENT;
                }
            }
        }
        printf("\n");
        while (base <= total_frames && frames[base] == ACKED) {
            base++;
        }
    }
    printf("-----------------------------------------\n");
    printf("Simulation Complete.\n");
    printf("Total original frames: %d\n", total_frames);
    printf("Total frames transmitted (including
retransmissions): %d\n", frames_sent_count);
    free(frames);
    return 0;
}
```

```
O/P
--- Selective Repeat Protocol Simulation ---
Enter window size: 4
Enter total number of frames to transmit: 10
-----------------------------------------

Sending Frame 1...
Sending Frame 2...
Sending Frame 3...
Sending Frame 4...
Acknowledgment for Frame 1 received.
Acknowledgment for Frame 2 received.
--> ERROR: Acknowledgment for Frame 3 lost or corrupted!
--> Action: Tagging Frame 3 for Selective Retransmission.
Acknowledgment for Frame 4 received.

Sending Frame 3...
Sending Frame 5...
Sending Frame 6...
--> ERROR: Acknowledgment for Frame 3 lost or corrupted!
--> Action: Tagging Frame 3 for Selective Retransmission.
Acknowledgment for Frame 5 received.
--> ERROR: Acknowledgment for Frame 6 lost or corrupted!
--> Action: Tagging Frame 6 for Selective Retransmission.

Sending Frame 3...
Sending Frame 6...
Acknowledgment for Frame 3 received.
Acknowledgment for Frame 6 received.

Sending Frame 7...
Sending Frame 8...
Sending Frame 9...
Sending Frame 10...
Acknowledgment for Frame 7 received.
Acknowledgment for Frame 8 received.
Acknowledgment for Frame 9 received.
--> ERROR: Acknowledgment for Frame 10 lost or corrupted!
--> Action: Tagging Frame 10 for Selective Retransmission.

Sending Frame 10...
Acknowledgment for Frame 10 received.
-----------------------------------------
Simulation Complete.
Total original frames: 10
Total frames transmitted (including retransmissions): 14
```

Stop and Wait

main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

int main() {
    int total_frames, frames_sent_count = 0;
    int current_frame = 1;
    srand(time(NULL));
    printf("--- Stop-and-Wait Protocol Simulation ---\n");
    printf("Enter total number of frames to transmit: ");
    scanf("%d", &total_frames);
    printf("-----------------------------------------\n\n");
    while (current_frame <= total_frames) {]
        printf("Sending Frame %d...\n", current_frame);
        frames_sent_count++;
        int ack_status = rand() % 5;
        sleep(1);

        if (ack_status != 0) {
            printf("Acknowledgment for Frame %d received.\n\n",
current_frame);
            current_frame++;
        } else {
            printf("--> ERROR: Acknowledgment for Frame %d lost
or corrupted!\n", current_frame);
            printf("--> Action: Timeout occurred. Retransmitting
Frame %d...\n\n", current_frame);
        }
    }
    printf("-----------------------------------------\n");
    printf("Simulation Complete.\n");
    printf("Total original frames: %d\n", total_frames);
```

```
    printf("Total frames transmitted (including
retransmissions): %d\n", frames_sent_count);
    return 0;
}
```

```
O/P
--- Stop-and-Wait Protocol Simulation ---
Enter total number of frames to transmit: 10
----------------------------------------

Sending Frame 1...
Acknowledgment for Frame 1 received.

Sending Frame 2...
Acknowledgment for Frame 2 received.

Sending Frame 3...
Acknowledgment for Frame 3 received.

Sending Frame 4...
Acknowledgment for Frame 4 received.

Sending Frame 5...
Acknowledgment for Frame 5 received.

Sending Frame 6...
--> ERROR: Acknowledgment for Frame 6 lost or corrupted!
--> Action: Timeout occurred. Retransmitting Frame 6...

Sending Frame 6...
Acknowledgment for Frame 6 received.

Sending Frame 7...
--> ERROR: Acknowledgment for Frame 7 lost or corrupted!
--> Action: Timeout occurred. Retransmitting Frame 7...

Sending Frame 7...
Acknowledgment for Frame 7 received.

Sending Frame 8...
Acknowledgment for Frame 8 received.

Sending Frame 9...
Acknowledgment for Frame 9 received.

Sending Frame 10...
Acknowledgment for Frame 10 received.

----------------------------------------
Simulation Complete.
Total original frames: 10
Total frames transmitted (including retransmissions): 12
```

DVR Algorithm

main.c

```
#include <stdio.h>
#define MAX_NODES 20
#define INF 9999
struct RoutingTable {
    unsigned dist[MAX_NODES];
    unsigned next_hop[MAX_NODES];
} rt[MAX_NODES];

int main() {
    int cost_matrix[MAX_NODES][MAX_NODES];
    int nodes, i, j, k, update_count = 0;
    printf("Enter the number of nodes: ");
    scanf("%d", &nodes);
    printf("\nEnter the cost matrix (use %d for infinity/no
direct link):\n", INF);
    for (i = 0; i < nodes; i++) {
        for (j = 0; j < nodes; j++) {
            scanf("%d", &cost_matrix[i][j]);
            if (i == j) {
                cost_matrix[i][i] = 0;
            }
            rt[i].dist[j] = cost_matrix[i][j];
            rt[i].next_hop[j] = j;
        }
    }
    do {
        update_count = 0;
        for (i = 0; i < nodes; i++) {
            for (j = 0; j < nodes; j++) {
```

```
                for (k = 0; k < nodes; k++) {
                    if (rt[i].dist[j] > cost_matrix[i][k] +
rt[k].dist[j]) {
                        rt[i].dist[j] = cost_matrix[i][k] +
rt[k].dist[j];
                        rt[i].next_hop[j] = k;
                        update_count++;
                    }
                }
            }
        }
    } while (update_count != 0);
    printf("\n--- Final Routing Tables ---\n");
    for (i = 0; i < nodes; i++) {
        printf("\nRouting Table for Router %d:\n", i + 1);
        printf("Destination\tNext Hop\tDistance\n");
        for (j = 0; j < nodes; j++) {
            printf("%d\t\t%d\t\t%d\n", j + 1, rt[i].next_hop[j]
+ 1, rt[i].dist[j]);
        }
    }
    return 0;
}
```

O/P

```
Enter the number of nodes: 3

Enter the cost matrix (use 9999 for infinity/no direct link):
0 2 9999
2 0 1
9999 1 0

--- Final Routing Tables ---

Routing Table for Router 1:
Destination     Next Hop        Distance
1               1               0
2               2               2
3               2               3

Routing Table for Router 2:
Destination     Next Hop        Distance
1               1               2
2               2               0
3               3               1

Routing Table for Router 3:
Destination     Next Hop        Distance
1               2               3
2               2               1
3               3               0
```

Leaky Bucket Algorithm

main.c

```
#include <stdio.h>

int main() {
    int incoming, outgoing, bucket_size, n, store = 0, dropped =
0;
    printf("Enter the bucket capacity: ");
    scanf("%d", &bucket_size);
    printf("Enter the outgoing rate (leak rate): ");
    scanf("%d", &outgoing);
    printf("Enter the number of time intervals to simulate: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("\n--- Time Interval %d ---\n", i + 1);
        printf("Enter the incoming packet size: ");
        scanf("%d", &incoming);
        if (incoming <= (bucket_size - store)) {
            store += incoming;
            printf("Accepted %d packets.\n", incoming);
        } else {
            dropped = incoming - (bucket_size - store);
            printf("Bucket overflow! Dropped %d packets.\n",
dropped);
            store = bucket_size;
        }
        if (store >= outgoing) {
            store -= outgoing;
            printf("Leaked %d packets. ", outgoing);
        } else {
            printf("Leaked %d packets. ", store);
            store = 0;
```

```
        }
        printf("Current bucket status: %d / %d\n", store,
bucket_size);
    }
    printf("\n--- Processing remaining packets in the bucket ---
\n");
    while (store > 0) {
        if (store >= outgoing) {
            store -= outgoing;
            printf("Leaked %d packets. ", outgoing);
        } else {
            printf("Leaked %d packets. ", store);
            store = 0;
        }
        printf("Current bucket status: %d / %d\n", store,
bucket_size);
    }
    printf("\nBucket is empty. Simulation complete.\n");
    return 0;
}
```

O/P

```
Enter the bucket capacity: 100
Enter the outgoing rate (leak rate): 20
Enter the number of time intervals to simulate: 3

--- Time Interval 1 ---
Enter the incoming packet size: 40
Accepted 40 packets.
Leaked 20 packets. Current bucket status: 20 / 100

--- Time Interval 2 ---
Enter the incoming packet size: 90
Bucket overflow! Dropped 10 packets.
Leaked 20 packets. Current bucket status: 80 / 100

--- Time Interval 3 ---
Enter the incoming packet size: 10
Accepted 10 packets.
Leaked 20 packets. Current bucket status: 70 / 100

--- Processing remaining packets in the bucket ---
Leaked 20 packets. Current bucket status: 50 / 100
Leaked 20 packets. Current bucket status: 30 / 100
Leaked 20 packets. Current bucket status: 10 / 100
Leaked 10 packets. Current bucket status: 0 / 100

Bucket is empty. Simulation complete.
```

FTP

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    char filename[256];
    FILE *file;
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <=
0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }
    if (connect(sock, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
    printf("Connected to the FTP Server.\n");
    printf("Enter the name of the file you want to download: ");
    scanf("%s", filename);
```

```
    send(sock, filename, strlen(filename), 0);
    int bytes_received = recv(sock, buffer, BUFFER_SIZE, 0);
    if (strncmp(buffer, "ERROR:", 6) == 0) {
        printf("%s\n", buffer);
    } else {
        char new_filename[300];
        snprintf(new_filename, sizeof(new_filename),
"downloaded_%s", filename);
        file = fopen(new_filename, "wb");
        if (file == NULL) {
            perror("Could not create local file");
            return -1;
        }
        fwrite(buffer, 1, bytes_received, file);
        while ((bytes_received = recv(sock, buffer, BUFFER_SIZE,
0)) > 0) {
            fwrite(buffer, 1, bytes_received, file);
        }
        printf("File downloaded successfully as '%s'.\n",
new_filename);
        fclose(file);
    }
    close(sock);
    return 0;
}
```

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};
    FILE *file;
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&address,
sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
    printf("FTP Server listening on port %d...\n", PORT);
    if ((new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }
    printf("Client connected.\n");
    recv(new_socket, buffer, BUFFER_SIZE, 0);
    printf("Client requested file: %s\n", buffer);
    file = fopen(buffer, "rb");
    if (file == NULL) {
        char *error_msg = "ERROR: File not found.";
        send(new_socket, error_msg, strlen(error_msg), 0);
        printf("File not found. Error message sent to client.
\n");
    } else {
        printf("Sending file...\n");
        int bytes_read;
        while ((bytes_read = fread(buffer, 1, BUFFER_SIZE,
file)) > 0) {
            send(new_socket, buffer, bytes_read, 0);
        }
        fclose(file);
        printf("File transfer complete.\n");
    }
    close(new_socket);
    close(server_fd);
    return 0;
```

```
}
O/P client
Connected to the FTP Server.
Enter the name of the file you want to download: test.txt
File downloaded successfully as 'downloaded_test.txt'.

O/P server
FTP Server listening on port 8080...
Client connected.
Client requested file: test.txt
Sending file...
File transfer complete.
```